



Master's thesis

Master's Programme in Computer Science

Reprogramming Large Language Models for Spatia-Temporal Traffic Flow Forecasting

Ke Zhang

May 22, 2025

FACULTY OF SCIENCE
UNIVERSITY OF HELSINKI

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki, Finland

Email address: info@cs.helsinki.fi

URL: <http://www.cs.helsinki.fi/>

Tiedekunta — Fakultet — Faculty		Koulutusohjelma — Utbildningsprogram — Study programme	
Faculty of Science		Master's Programme in Computer Science	
Tekijä — Författare — Author			
Ke Zhang			
Työn nimi — Arbetets titel — Title			
Reprogramming Large Language Models for Spatia-Temporal Traffic Flow Forecasting			
Ohjaajat — Handledare — Supervisors			
Prof. Mikko. Koivisto			
Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages	
Master's thesis	May 22, 2025	38 pages, 6 appendix pages	
Tiivistelmä — Referat — Abstract			
Traffic flow forecasting is a critical component of intelligent transportation systems. Accurate traffic forecasting can optimize traffic management and public transit scheduling, making it a crucial component of the intelligent transportation system. However, existing large-scale traffic model-based traffic prediction models lack the global modeling and transfer capabilities of large language models. To address these challenges, we propose Spatio-Temporal series forecasting, a reprogramming-based large language model, named STF-GPT2, which leverages the transfer learning capability of a pre-trained GPT2 model for traffic data analysis. We design an effective multi-modal alignment strategy that combines convolutional fusion and cross-attention mechanisms to align three distinct modalities: temporal, spatial, and natural language representations. Furthermore, we introduce the Prompt-as-Prefix (PaP) method, which provides task-oriented instructions to the LLM in the form of natural language prompts. Extensive experiments conducted on two real-world traffic datasets demonstrate the superior performance of our model over several state-of-the-art approaches in long-series forecasting tasks and demonstrates strong effectiveness in few-shot and zero-shot scenarios.			
ACM Computing Classification System (CCS) Information system → Information systems applications → Spatial-temporal systems Computing methodologies → Artificial intelligence			
Avainsanat — Nyckelord — Keywords			
algorithms, large language model, traffic flow forecasting, Spatio-Temporal series forecasting			
Säilytyspaikka — Förvaringsställe — Where deposited			
Helsinki University Library			
Muita tietoja — övriga uppgifter — Additional information			
Networking study track			

Contents

1	Introduction	1
2	Background	4
2.1	Modality Alignment	4
2.2	Model Reprogramming	5
2.3	Few-Shot and Zero-Shot	5
3	Methods	7
3.1	Problem Definition	7
3.2	Overview	8
3.3	Spatio-Temporal Embedding Layer	8
3.4	Embedding Fusion Layer	10
3.5	Modality Alignment Layer	11
3.6	Prompt-as-Prefix (PaP) Layer	13
3.7	Frozen Pre-trained LLM Layer	15
3.8	Convolutional Regression Layer	16
3.9	Algorithm Framework	17
4	Experiment	20
4.1	Dataset	20
4.2	Baseline	21
4.3	Evaluation Metrics	21
4.4	Implementations	22
4.5	Model Optimization	23
4.6	Ablation Study of Different Components	25
5	Results	28
5.1	Comparative Experiments	28
5.2	Few-Shot Forecasting	29

5.3 Zero-Shot Forecasting	31
6 Conclusions	34
Bibliography	35
A Experiment Result Data	i
B Reproduce Project	i
C Use of Language and Writing Assistance Tools	i

1 Introduction

Spatio-Temporal series are an extension of time series into the spatial domain, it refers to data that exhibit both temporal and spatial dependencies. Not only reflects the dynamic evolution of variables over time, but it also captures the interactions between different spatial locations. Compared to traditional time series, which only model temporal evolution, Spatio-Temporal series are more complex. Their modeling and forecasting tasks must jointly consider temporal dynamics and spatial interactions, which is a typical problem of multidimensional correlation modeling. This type of series modeling is widely applied in fields such as traffic flow forecasting, weather forecasting [23], and epidemic spread analysis [4].

Traffic flow forecasting refers to predicate the level of traffic usage in various urban regions during specific future time intervals using historical data [6]. Accurate traffic prediction can optimize traffic management and public transit scheduling, making it a crucial component of intelligent transportation systems [1]. For example, forecasting bike-sharing flow can guide precise station deployments [15]; forecasting bus or taxi demand can improve the efficiency of vehicle dispatching in public transit systems [8][26].

Early traffic forecasting relied primarily on classical time series models such as the Kalman filter and ARIMA [1][11], which can fit time dependencies well but fail to capture the more complex Spatio-Temporal dependencies inherent in traffic flow data. With the widespread adoption of deep learning, more sophisticated neural network architectures have been designed to decode traffic flow data, using sequential, parallel, or single-feature approaches to handle spatial and temporal embeddings of traffic data. However, mainstream models such as GCNs [29], RNNs [32], and CNNs share several common shortcomings [22]: They typically perform only local spatial modeling and struggle to capture dynamic changes and long-range series dependencies; They lack cross-domain generalization ability, often requiring retraining or fine-tuning for new tasks; As model architectures evolve, existing traffic prediction models have become increasingly large and complex.

Recent progress has demonstrated that large language models can achieve remarkable performance in time series forecasting tasks [9][33]. Compared with traditional deep learning models, LLMs offer several distinct advantages in this context:

- **Global Dependency Modeling:** LLMs leverage the global attention mechanism of Transformers, which naturally suits capturing long-range and remote dependencies, improving long-series modeling accuracy.
- **Transfer Learning:** Pre-trained LLMs exhibit strong transfer learning capabilities, enabling effective few-shot and zero-shot forecasting even in data-scarce scenarios.
- **Multimodal Knowledge:** LLMs can encode time series as textual tokens or integrate them with other modalities (e.g., speech, images) for multimodal forecasting tasks.
- **Enhanced Reasoning:** LLMs can incorporate task prompts to guide semantic-level understanding and reasoning, enabling logical inference and pattern recognition in complex dynamic systems.

However, focusing solely on the temporal dimension is insufficient; we must also account for inter-spatial correlations in traffic flow forecasting [5]. Simply concatenating time series with spatial feature vectors or merging them naively ignores their inherent structural differences, making it difficult to effectively model Spatial-Temporal dependencies. Ensuring temporal and spatial feature alignment, where each time point is matched with its accurate spatial state and corresponding historical trends and spatial neighborhood effects is a significant challenge. LLMs operate on discrete natural language tokens, while time and spatial series are inherently continuous [27]. Aligning Spatio-Temporal modality data with the language modality of LLMs remains non-trivial. Additionally, pre-trained LLMs typically lack the knowledge and reasoning capabilities to interpret tokenized Spatio-Temporal series aligned to the language domain. Thus, activating LLM generalization ability in Spatio-Temporal forecasting tasks presents another major challenge.

To address these challenges, we propose STF-GPT2, a reprogramming framework that leverages the transfer learning capability of large language models for Spatio-Temporal series forecasting while keeping the backbone of a pre-trained GPT2 model unchanged. Within the STF-GPT2 framework, we design a dedicated Spatio-Temporal embedding layer to encode the tokens of temporal and spatial features separately. This embedding layer comprehensively captures spatial locations and global temporal patterns and uses convolution-based fusion to align the two modalities.

To make Spatio-Temporal series more compatible with the LLMs text-based modality, we adopt a cross-attention mechanism to achieve modality alignment. Furthermore, to

activate the LLMs generalization ability in Spatio-Temporal forecasting, we introduce the Prompt-as-Prefix (PaP) method, which provides task-oriented instructions to the LLM in the form of natural language prompts. Finally, the LLM output is passed through convolutional regression layers to produce the prediction results.

We conduct extensive experiments on two public traffic flow datasets, and results show that our model outperforms several state-of-the-art Spatio-Temporal forecasting models in long-series forecasting tasks and demonstrates strong effectiveness in few-shot and zero-shot scenarios. The contributions of the paper are summarized below:

- We propose a novel large language model based on reprogramming, named STF-GPT2, for Spatio-Temporal traffic forecasting. By leveraging a pre-trained GPT-2 backbone, the model fully exploits the global modeling and transfer capabilities of large language models for traffic data analysis.
- We design an effective multimodal alignment strategy that combines convolutional fusion and cross-attention mechanisms to align three distinct modalities: temporal, spatial, and natural language representations.
- To further activate the forecasting potential of the pre-trained model, we introduce the Prompt-as-Prefix (PaP) mechanism, which provides task-specific instructions. This enables the model to better understand the semantic intent of traffic flow forecasting tasks across different domains.
- Extensive experiments conducted on two real-world traffic datasets demonstrate the superior performance of STF-GPT2 over strong baselines, in both standard forecasting settings, few-shot and zero-shot scenarios.

2 Background

2.1 Modality Alignment

Modality alignment refers to the process of mapping heterogeneous input sources into a semantically consistent representation space, thereby enabling effective fusion and reasoning across modalities such as text, images, and time series. In Spatio-Temporal forecasting tasks, modality alignment is particularly crucial due to the inherent differences in representation formats: temporal features are continuous and sequential, spatial representations are structured and often graph-based, while language tokens are discrete and symbolic. These discrepancies pose significant challenges when adapting pre-trained large language models to Spatio-Temporal forecasting scenarios.

To address these challenges, previous research has proposed several modality alignment strategies. Joint representation learning methods, such as CLIP, map all modalities into a shared latent space to achieve unified cross-modal representations [19]. Alternatively, cross-attention mechanisms, as employed in Perceiver IO, enable models to dynamically attend to and interact across modalities [7]. More recently, modality reprogramming techniques such as Voice2Series have shown that non-textual data can be encoded into language-like token series [31], effectively aligning heterogeneous modalities with minimal modification to the underlying model architecture.

In this work, we propose a hybrid alignment strategy that combines convolutional fusion and cross-attention mechanisms to jointly align spatial, temporal, and textual modalities. This design preserves the dynamic correlations between time steps and spatial locations, and establishes meaningful associations between Spatio-Temporal series and textual tokens. As a result, the LLM backbone is able to better perceive and understand the underlying structure of Spatio-Temporal data, ultimately enhancing the accuracy and generalization capability of the forecasting model.

2.2 Model Reprogramming

Traditional series forecasting models are typically tailored for specific tasks and domains. For instance, ARIMA is designed for specific time series prediction [11], while models like GAT and GCN are specifically developed for Spatial-Temporal forecasting [24]. Although these models perform well within their respective domains, their architectural and design constraints limit their ability to generalize across different data types or tasks, resulting in poor adaptability and transferability.

To overcome these limitations, recent research has increasingly shifted toward more general and transferable model paradigms, inspired by the success of pretraining fine-tuning frameworks in natural language processing and computer vision [20][10]. By leveraging self-supervised pretraining on large-scale, heterogeneous time series data, models can learn more universal temporal representations, enabling more efficient adaptation to downstream tasks with minimal fine-tuning. One emerging approach within this paradigm is Model Reprogramming. A representative example is the Time-LLM framework, which adapts pre-trained large language models for time series tasks. Instead of modifying the model architecture, Time-LLM reprograms the input space by converting raw numerical time series into natural language-like tokens, series embedding the values as descriptive textual input. These are combined with task-specific natural language prompts to activate the contextual understanding and reasoning capabilities of LLMs, such as GPT-2 or Bert. Crucially, this reprogramming method requires no changes to the model structure or parameters, allowing LLMs to handle time series forecasting without fine-tuning [9]. This dramatically reduces deployment and training costs while avoiding overfitting issues commonly associated with fine-tuning large models on small datasets.

2.3 Few-Shot and Zero-Shot

Zero-Shot learning was first proposed by Larochelle et al. (2008)[12]. Its core idea is to enable a model to directly perform predictions on new tasks or categories that were not seen during the training phase. Specifically, Zero-Shot learning requires the model to leverage the knowledge acquired during training and its inherent generalization ability to infer unseen data distributions or task types, without any fine-tuning or additional training on the new tasks. This capability is particularly important for practical scenarios where tasks are diverse and labeled data is scarce. In this work, we design Zero-Shot

prediction experiments to evaluate the performance of model in transferring directly from one traffic task (e.g., taxi pick-up flow prediction) to another related task within the same city (e.g., taxi drop-off flow prediction) without fine-tuning, thus validating the cross-task generalization ability of model.

Few-Shot learning focuses on how a model can quickly adapt to and complete new tasks with only a very limited number of labeled samples. This learning paradigm was systematically proposed by Vinyals et al. (2016) in their work [25], aiming to enable models to learn new classes or tasks from only a few examples, overcoming the dependency on large labeled datasets typical in traditional machine learning methods. Few-Shot learning usually relies on transfer learning and meta-learning abilities to improve learning efficiency under scarce data conditions. To simulate the real-world scenario of limited data availability, we trim the original training set to retain only 10% of the samples and test the model’s adaptation and prediction capabilities in this low data regime.

3 Methods

In this chapter, we introduce our reprogrammed large language model for Spatio-Temporal traffic flow forecasting, STF-GPT2. We begin by clearly defining the traffic flow forecasting task. Based on this problem definition, we then present the architecture of our proposed model. For ease of explanation, we divide the entire framework into six functional components: the Spatio-Temporal embedding layer, the Spatio-Temporal fusion layer, the modality alignment layer, the prompt-as-prefix layer, the frozen pre-trained LLMs layer, and the convolutional regression layer.

3.1 Problem Definition

We represent the historical traffic flow data as $\mathbf{X}_P = [\mathbf{X}_{t-P+1}, \mathbf{X}_{t-P+2}, \dots, \mathbf{X}_t]$, where $\mathbf{X}_P \in \mathbb{R}^{P \times N \times F}$, t represents the current time point in the time series. P denotes the length of historical time steps, N represents the number of sensor locations, and F indicates the number of collected features. For example, if $F = 100$, it means that the sensor records a traffic flow of 100 units during the time interval of P time steps.

The goal of traffic flow prediction is to estimate future traffic flow as $\hat{\mathbf{Y}}_Q = [\hat{\mathbf{Y}}_{t+1}, \hat{\mathbf{Y}}_{t+2}, \dots, \hat{\mathbf{Y}}_{t+Q}]$, where $\hat{\mathbf{Y}}_Q \in \mathbb{R}^{Q \times N \times F}$, and Q denotes the number of future time steps to be predicted.

The task of Spatio-Temporal traffic flow series forecasting is to learn a function $f(\cdot)$ that understands the input $\mathbf{X}_P$, and accurately forecasts the traffic flow over the next Q time steps, i.e., $\hat{\mathbf{Y}}_Q$. The overall objective is to minimize the error between the predicted values and the ground truth values, as defined:

$$\hat{\mathbf{Y}}^Q = f(\mathbf{X}_P) \quad (3.1)$$

$$\mathcal{L} = \text{Loss}(\hat{\mathbf{Y}}^Q, \mathbf{Y}_Q) \quad (3.2)$$

Where $\mathcal{L}$ denotes the loss function, which is defined as the Mean Squared Error (MSE).

3.2 Overview

As illustrated in Fig. 3.1, the Spatial-Temporal Series Large Language Model framework integrates several key components: the Spatial-Temporal embedding layer, Embedding fusion layer, modality alignment layer, Prompt-as-Prefix (PaP) layer, frozen pre-trained LLM layer, and convolutional regression layer. The input historical traffic flow data is denoted as $X_P \in \mathbb{R}^{P \times N \times F}$, where P is the number of historical time steps, N is the number of sensor locations, and F represents the number of collected features (e.g., traffic flow). First, the Spatial-Temporal embedding layer extracts token embeddings from P time steps, including fine-grained token embeddings $E_F \in \mathbb{R}^{N \times D}$, hour-level temporal embeddings $E_h \in \mathbb{R}^{N \times D}$, day-level temporal embeddings $E_d \in \mathbb{R}^{N \times D}$, and spatial embeddings $E_S \in \mathbb{R}^{N \times D}$. These embeddings are fused in the embedding fusion layer to produce a unified representation $H_F \in \mathbb{R}^{N \times D'}$. A pre-trained embedding layer $E \in \mathbb{R}^{V \times D}$ is then loaded. To reduce dimensionality before performing cross-attention, a linear projection is applied to E' , producing $E_V \in \mathbb{R}^{V' \times D}$ and $E_K \in \mathbb{R}^{V' \times D}$, which serve as the Value and Key in the attention mechanism, respectively. In the modality alignment layer, H_F serves as the Query and is aligned with E_V and E_K through a cross-attention mechanism to obtain the aligned representation H_C . Simultaneously, a pre-defined prompt text is processed into embeddings denoted as E_{PaP} . The concatenated output of H_C and E_{PaP} forms the input representation H_{input} , which is fed into the frozen pre-trained LLM to extract deeper contextual features, resulting in the output H_{output} . Before H_{output} is passed into the convolutional regression layer, the extra prefix tokens are removed. The final output predicts traffic flow $\hat{\mathbf{Y}}_Q \in \mathbb{R}^{Q \times N \times F}$, over the next Q time steps across N sensors.

3.3 Spatio-Temporal Embedding Layer

The goal of the Spatio-Temporal embedding layer is to transform historical traffic data into Spatio-Temporal representations by extracting hourly-level temporal embeddings, daily-level temporal embeddings, spatial embeddings, and traffic feature embeddings from X_p .

For the temporal embedding layer, prior studies have demonstrated that incorporating dual-resolution temporal embeddings, specifically the hour of day and day of week, can effectively capture both intra-day and weekly periodic patterns in traffic flow forecasting tasks [14][28][17][30]. Take Beijing's traffic congestion as an example – typical congestion periods occur at 8:00 AM-10:00 AM and 4:00 PM-7:00 PM, mainly from Monday to Friday

(working days). This pattern is primarily due to private vehicles commuting during rush hours [34]. To capture global traffic flow variation patterns, it is essential to model features from both temporal resolutions.

We use linear layers to encode the input data $X_{hod} \in \mathbb{R}^{N \times T_h}$ and $X_{dow} \in \mathbb{R}^{N \times T_w}$, generating the hour-level temporal embedding $E_h \in \mathbb{R}^{N \times D}$ and the day-level temporal embedding $E_d \in \mathbb{R}^{N \times D}$, respectively. Additionally, the linear layers possess certain learning capabilities, enabling the model to learn patterns of temporal influence. The corresponding embedding algorithms are as follows:

$$E_h = W_h(X_{hod}) \quad (3.3)$$

$$E_d = W_d(X_{dow}) \quad (3.4)$$

$$E_T = E_h + E_d \quad (3.5)$$

where W_h and W_d are learnable parameters for the hour-of-day and day-of-week embeddings, respectively. Since E_h and E_d have the same shape and are aligned in temporal resolution, they are fused by element-wise addition to form the final temporal embedding $E_T \in \mathbb{R}^{N \times D}$.

Inspired by the adaptive spatial modeling strategy proposed in Graph WaveNet [28], we adopt a weak spatial embedding approach to capture spatial dependencies without relying on an explicit adjacency matrix. Specifically, each spatial node is assigned a unique, randomly initialized and learnable embedding vector, which is optimized via backpropagation during training. This spatial embedding layer functions analogously to position encoding in Transformers or word embeddings in NLP, encoding node identity and implicit spatial structure.

Our method enables the model to learn a latent, data-driven spatial relationship among nodes, thereby assisting downstream layers in modeling spatial dependencies. The feasibility and effectiveness of such weak structural embeddings have also been validated in recent large language model-based spatial-temporal frameworks, such as ST-LLM [16], STG-LLM [17], and GPT4TFP [30]. Formally, we define a trainable spatial embedding matrix as:

$$E_S = \text{SpatEmbed}(N) \in \mathbb{R}^{N \times D} \quad (3.6)$$

where N is the number of nodes and D is the embedding dimension. Each node i is mapped to a spatial embedding vector $\mathbf{e}_i \in \mathbb{R}^D$, such that: $\mathbf{e}_i = E_S[i, :]$, $i = 1, 2, \dots, N$.

In traffic forecasting tasks, each node (e.g., sensor or road location) collects multi-dimensional traffic features (such as speed, flow, and occupancy) across multiple time steps. To transform the original sequential data into a unified token representation, we design a lightweight pointwise convolutional embedding layer. This layer operates solely on the feature dimension while preserving the original spatio-temporal structure of the input data. The resulting embeddings can seamlessly interface with subsequent spatio-temporal alignment, ensuring both semantic integrity and structural consistency.

$$E_F = PConv(\mathbf{X}_P; \boldsymbol{\theta}_F) \in \mathbb{R}^{N \times D} \quad (3.7)$$

where E_F is the token embedding, D is the unified hidden dimension, $PConv$ denotes the pointwise convolution operation, and $\boldsymbol{\theta}_F$ represents the learnable parameters of the convolution.

3.4 Embedding Fusion Layer

We obtain three different types of embeddings from the spatial-temporal embedding layer: temporal embedding E_T , spatial embedding E_S , and token embedding E_F . These three embeddings originate from heterogeneous sources: temporal features are periodic, spatial features are structured, and tokens are continuous or sparse. If we directly concatenate or add these embeddings, the model may struggle to extract meaningful joint representations due to the entangled and heterogeneous nature of the input. Moreover, the embeddings operate on different scales, and such inconsistencies can lead to instability in the joint embedding space and make model training more difficult. This simple concatenation approach has been adopted in works such as STEFT [3], where spatial, temporal, and token embeddings are concatenated along the feature dimension before being fed into the Transformer. However, this method treats each modality equally and does not explicitly model the interaction weights among heterogeneous embeddings.

If each embedding is separately used in cross-attention with the LLM, the joint interactions and co-representations among them may be lost. Therefore, we apply a convolutional fusion method. Pointwise convolution is performed across the feature dimension (i.e., the concatenated token/time/space channels), enabling the model to learn a weighted combi-

nation of token, temporal, and spatial features and capture local combination patterns. This learnable fusion mechanism allows the model to adaptively emphasize important modalities and their combinations, improving the concatenate embedding quality. Furthermore, this fusion process compresses the concatenated representation of $3D$ dimensions to the target dimension D' , which aligns with the input dimension of the pretrained LLM embedding layer, facilitating subsequent cross-attention operations. Such convolutional fusion strategies have shown effectiveness in recent models like STFGCN [13], which dynamically fuse temporal and spatial modalities via learnable convolutions to capture complex Spatio-Temporal dependencies. The fusion operation is defined as:

$$\mathbf{H}_F = FConv(E_F || E_T || E_S; \theta_f) \quad (3.8)$$

Where $E_F \in \mathbb{R}^{N \times D}$ represents the token embedding, $E_S \in \mathbb{R}^{N \times D}$ represents the spatial embedding, and $E_T \in \mathbb{R}^{N \times D}$ represents the temporal embedding. After concatenation, the combined embedding is $[E_F || E_S || E_T] \in \mathbb{R}^{N \times 3D}$, this representation is passed through the fusion convolution $FConv(\cdot)$ module to produce the fused embedding: $\mathbf{H}_F \in \mathbb{R}^{N \times D}$ where D is the input dimension of the pretrained LLM embedding layer.

3.5 Modality Alignment Layer

The pre-trained LLM is trained on natural language data and expects input in the form of tokenized text embeddings, where the semantic units are linguistic structures such as words, phrases, or sentences. In contrast, $\mathbf{H}_F$ represents the fused representation of Spatio-Temporal data, whose organizational form is numerical signals derived from continuous series, such as trends, traffic flow, and spatial correlations. Although both $\mathbf{H}_F \in \mathbb{R}^{N \times D}$ and the token embedding space of the LLM $\mathbb{R}^{T \times D}$ share the same dimensionality, they differ greatly in semantic distribution. Therefore, dimensional alignment alone cannot ensure that the LLM can correctly interpret the meaning of $\mathbf{H}_F$.

To align the modality of Spatio-Temporal series with that of natural language and activate the LLM ability to understand and reason over such series, we adopt a cross-attention mechanism to achieve semantic alignment. For example, each upward or downward trend in a traffic patch can be used as a query to search for the most semantically matching token in the language space, such as "upward" or "drop." This process maps patterns of numerical variation in the series to interpretable natural language representations, which is the core

of cross-attention. However, in practice, there is often no explicit one-to-one correspondence between Spatio-Temporal features and natural language tokens describing trends. Instead, the model adaptively combines multiple different tokens in a multi-directional manner to achieve semantic fusion. Before performing cross-attention with the pre-trained embedding layer, an important issue must be considered. Taking the pre-trained embedding layer of GPT-2 as an example, the embedding matrix $\mathbf{E} \in \mathbb{R}^{50257 \times 768}$ contains embeddings for 50,257 vocabulary tokens. However, the number of tokens relevant to traffic flow series prediction is far smaller than 50,257. Retaining the full size of $\mathbf{E}$ during each cross-attention operation would cause a substantial and unnecessary computational overhead. Therefore, we employ a linear projection layer to map the original embedding $\mathbf{E}$ to a smaller, more compact set $\mathbf{E}' \in \mathbb{R}^{V' \times D}$, where $V' \ll 50257$ and D is the embedding dimension of the LLM. This dimensionality reduction strategy is inspired by the ST-LLM framework [9], which applies a similar linear projection to compress the pretrained vocabulary embedding space. By projecting high-dimensional token representations into a more compact subspace, the model not only reduces memory and computational overhead but also promotes more efficient alignment between task-specific tokens and the pretrained language model. During training, through backpropagation, representative tokens within the reduced vocabulary V' are learned by linearly probing $\mathbf{E}$, gradually forming a concise set of language prototypes relevant to traffic flow prediction:

$$E' = \text{LinearProj}(E) \in \mathbb{R}^{V' \times D}, \quad \text{where } V' \ll V, \quad (3.9)$$

where V denotes the vocabulary size of the pre-trained LLM embedding layer, and D denotes the embedding dimension of the LLM.

After performing a linear mapping on the pre-trained embedding matrix $\mathbf{E}$, we obtain a compact set of language prototypes $\mathbf{E}' \in \mathbb{R}^{V' \times D}$. We use $\mathbf{H}_F$ as the query matrix $\mathbf{Q}$, and $\mathbf{E}'$ serves as both the key matrix $\mathbf{K}$ and the value matrix $\mathbf{V}$. Three learnable linear mappings are defined as follows:

$$E_Q = W_Q(H_F), \quad E_K = W_K(E'), \quad E_V = W_V(E'), \quad (3.10)$$

where $W_Q, W_K, W_V \in \mathbb{R}^{D \times d}$ denote the linear projection weights, and d is the projection dimension.

The cross-attention is then computed as:

$$H_C = \text{CrossAttention}(E_Q, E_K, E_V) = \text{softmax}\left(\frac{E_Q E_K^\top}{\sqrt{d}}\right) E_V \quad (3.11)$$

where $H_C \in \mathbb{R}^{N \times D}$ represents the aligned spatio-temporal and language fused embedding.

3.6 Prompt-as-Prefix (PaP) Layer

Prompt-as-Prefix text:

<|start_prompt|>

Dataset Description: This dataset records {domain} from {number of nodes} zones, including hourly and daily {domain} traffic information.

Input Statistics:

Average pickup flow: {avg_flow} per time step.

Max pickup flow: {max_flow}

Min pickup flow: {min_flow}

Median pickup flow: {median_flow}

...

Task Description: Given a series of the last {T} time steps for all zones, predict the pick up flow for the next {T} time steps.

<|end_prompt|>

Table 3.1: Prompt format used in traffic forecasting task. Curly braces {} indicate variables filled at runtime.

Although we obtain semantically aligned Spatio-Temporal representations $\mathbf{H}_C \in \mathbb{R}^{N \times D}$ via a Cross-Attention mechanism, feeding them directly into a pre-trained large language model often yields unsatisfactory performance, as observed in our ablation experiment on Section 4.6. This primarily stems from the nature of LLMs, which are pre-trained on massive corpora of structured natural language and therefore expect inputs in human-readable text form. In contrast, raw Spatio-Temporal embeddings lack linguistic structure and semantic grounding, making them difficult for LLMs to interpret and reason upon. As a result, without the support of explicit language prompts, such inputs fail to activate the LLM latent knowledge and cannot effectively guide it to perform downstream forecasting tasks.

A common strategy is **Numerical-to-Text** Prompting, where spatio-temporal series are converted into natural language descriptions and embedded as textual context for LLMs. This approach has been explicitly adopted in recent work such as AutoTimes [18], where

multivariate time series are transformed into descriptive text sequences (e.g., "value1 at t1 is 3.5, value2 at t1 is 7.9...") and autoregressively predicted by LLMs. However, this method is only feasible when the input series are simple and exhibit stable patterns. Spatio-temporal series, by nature, are high-dimensional and structurally complex. Compressing such rich information into a single piece of natural language without significant information loss is extremely challenging, if not impossible. Moreover, including too many detailed statistical descriptions of spatio-temporal features can lead to overly long prompts, easily exceeding the maximum context window of LLMs and limiting their practical effectiveness.

To address these challenges, we adopt a **Prompt-as-Prefix (PaP)** strategy, which places language instructions specific to the traffic forecasting task as a prefix before the numerical representations (e.g., the semantically aligned H_C). This design effectively activates the LLM’s inherent language knowledge and enhances its predictive reasoning ability. The core idea of PaP is to prepend a carefully crafted, semantically rich natural language prompt before the input embeddings. The prompt typically includes components such as task description, temporal context, or spatial semantics. This approach has been validated in prior work, notably Time-LLM [9], where a similar PaP mechanism is employed to guide LLMs for time series forecasting. Their results demonstrate that prefixing task-specific natural language prompts can significantly improve forecasting performance, especially when LLMs are reprogrammed for numerical prediction tasks.

Furthermore, the design of *domain-specific and task-oriented* prompt templates plays a crucial role in the effectiveness of the PaP approach. Empirical evidence suggests that selecting *meaningful statistical parameters*—such as mean, variance, trend, seasonality indicators, and spatial correlations can provide stronger priors about the Spatio-Temporal patterns, and helping the LLM better model complex dependencies [9]. These statistical descriptors, when properly embedded into the prompt, enrich the semantic content and offer high-level summaries of the input data. At the same time, prompt brevity remains essential: too long prompts can substantially increase the tokenized input length of LLMs, leading to higher computational costs, memory usage, and potential degradation in model efficiency. Thus, a balance must be struck between the richness of statistical content and the conciseness of prompt design. The prompt typically includes the following components:

- **Dataset Description:** Provides background knowledge about the traffic flow dataset, which helps activate domain-specific prior knowledge within the LLM;
- **Task Description:** Clearly articulates the goal of the forecasting task, guiding the

model’s attention toward the desired output;

- **Input Statistics:** Quantitatively summarizes global characteristics of the Spatio-Temporal data, such as value range, trends, and fluctuations, offering the LLM a structured overview to aid its understanding of temporal dynamics.

Before concatenating the prompt text as a prefix with the Spatio-Temporal representation H_C , the textual prompt must first be transformed into embedding vectors. Specifically, we denote the embedded prompt as:

$$E_{PaP} = Embed(\mathbf{txt}) \quad (3.12)$$

Where $E_{PaP} \in \mathbb{R}^{l \times D}$, with l representing the tokenized length of the prompt text and D denoting the embedding dimension of the LLM. Where $Embed(\cdot)$ refers to the LLM textual encoder, which includes a pre-trained tokenizer followed by a pre-trained embedding layer. The input $\mathbf{txt}$ corresponds to the natural language prompt used in the task.

3.7 Frozen Pre-trained LLM Layer

To integrate the numerical Spatio-Temporal representation with the language prompt, we concatenate the prompt embedding $\mathbf{E}_{PaP} \in \mathbb{R}^{l \times D}$ and the aligned Spatio-Temporal feature $\mathbf{E}_C \in \mathbb{R}^{N \times D}$ along the series (token) dimension. As both share the same embedding dimension D , this concatenation is implemented as a simple row-wise stacking:

$$E_{input} = Concat(E_{PaP}, E_C) \in \mathbb{R}^{(l+N) \times D} \quad (3.13)$$

Where $Concat(\cdot)$ denotes vertical concatenation along the series axis. The resulting E_{input} serves as input to the frozen LLM.

Recent studies have shown that pretrained language models can achieve strong generalization and prediction performance in long-term time series forecasting [9][21][35]. Remarkably, this holds even when the models are fully frozen that is, all Transformer layers, including multi-head attention (MHA) and feed-forward networks (FFN), remain non-trainable.

Inspired by these findings, we introduce a similar freezing strategy for Spatio-Temporal forecasting tasks. Our goal is to preserve the rich language knowledge embedded in large language models, while avoiding the high computational cost of full fine-tuning.

In this work, we feed the input embedding E_{input} , which fuses language prompts with Spatio-Temporal representations, into a frozen LLM (i.e., Fully Pretrained Transformer, FPT) for contextual feature modeling. The resulting output representation is given by:

$$E_{output} = FptLLM(E_{input}) \in \mathbb{R}^{(l+N) \times D} \quad (3.14)$$

where l is the number of tokens in the language prefix prompt, N is the number of Spatio-Temporal tokens, and D is the embedding dimension.

To further extract Spatio-Temporal semantics for prediction, we retain only the last N tokens from the LLM output, corresponding to the aligned Spatio-Temporal input. The first l tokens, which are derived from the language prompt E_{PaP} , primarily serve to provide contextual task guidance and do not contribute directly to the prediction.

Thus, we truncate the output by removing the first l tokens and obtain the final Spatio-Temporal representation:

$$E'_{output} = E_{output}[:, l :, :] \in \mathbb{R}^{N \times D} \quad (3.15)$$

This final representation E'_{output} is then passed into the convolutional regression head to generate the multi-step traffic predictions.

3.8 Convolutional Regression Layer

Despite the fact that H'_{output} encodes rich Spatio-Temporal contextual semantics, it remains a high-dimensional abstract representation and does not directly correspond to the actual numerical values required for traffic forecasting, such as the future traffic flow at each spatial location. Therefore, a mapping mechanism is needed to transform this latent representation into explicit prediction outputs $\hat{\mathbf{Y}}_Q \in \mathbb{R}^{Q \times N \times C}$, where Q is the number of prediction steps, N is the number of spatial nodes, and C is the number of target channels (e.g., $C = 1$ in univariate traffic flow forecasting).

To this end, we introduce a two-dimensional regression convolution module (RConv). The 2D convolution is particularly powerful in local modeling, as it simultaneously captures dependencies along both the temporal dimension (Q) and the spatial dimension (N). Specifically, it extracts short-term temporal evolution patterns across successive time steps; and spatial interactions among neighboring regions. This is especially beneficial in traffic forecasting tasks, where local temporal continuity and spatial dependencies are critical. Moreover, the 2D convolution enables dimensionality reduction from the LLM

hidden size D to the target output size C , effectively serving as a decoder that transforms abstract representations into concrete predictions.

The prediction is formally defined as:

$$\hat{Y}_Q = RConv(H'_{output}; \theta_r) \quad (3.16)$$

where $\hat{Y}_Q \in \mathbb{R}^{Q \times N \times C}$, θ_r denotes the learnable parameters of the regression convolution layer.

3.9 Algorithm Framework

In order to clearly illustrate the overall workflow and key components of the proposed STF-GPT2 model, we present Algorithm 1 which summarizes the training procedure. This algorithmic framework provides a step-by-step outline of how spatial-temporal embeddings are computed, fused, and integrated with natural language prompts through the Prompt-as-Prefix (PaP) strategy. It also details the subsequent projection, cross-attention mechanism with the frozen pre-trained Transformer layers, and the prediction update steps.

Presenting the algorithm framework helps reader to systematically understand the flow of data and computation within the model, clarifies the interaction between different modules, and highlights the design choices for embedding fusion and prompt integration. Furthermore, this concise summary facilitates reproducibility and provides a reference blueprint for implementing the model in practice.

Algorithm 1: The STF-GPT2 Framework**Input:** Traffic feature X_P from the past P timesteps, and all model configures**Output:** Trained STF-GPT2

```

1 for each epoch do
2   Shuffle training data;
3   for each batch  $X_P$  in training data do
4     // Compute spatial-temporal embedding
5      $E_T, E_S, E_F \leftarrow$  Equations (3.3), (3.4), (3.5), (3.6), (3.7) with  $X_P$ ;
6     // Fuse embeddings
7      $H_F \leftarrow$  Equation (3.8) with  $E_T, E_S, E_F$  ;
8     // PaP embedding
9      $E_{PaP} \leftarrow$  Equation (3.12) with txt;
10    // Projection embedding layer
11     $E' \leftarrow$  Equation (3.9) with  $E$ ;
12    // Cross-attention
13     $H_C \leftarrow$  Equation (3.10), (3.11) with  $H_F, E'$ ;
14    // Concatenate Fusion embedding and PaP embedding
15     $H_{input} \leftarrow$  Equation (3.13) with  $E_{PaP}$  and  $H_C$ ;
16    for  $i \leftarrow 1$  to 2 do
17      // Pre-trained Transformer layer processing
18       $H_{output} \leftarrow$  Equation (3.14) with  $H_{input}$ ;
19      // Remove PaP tokens
20       $H'_{output} \leftarrow$  Equation (3.15) with  $H_{output}$ ;
21      // Predict output
22       $\hat{Y}_S \leftarrow$  Equation (3.16) with  $H'_{output}$ ;
23      // Update parameters via adam optimizer
24      Minimize loss (Equation (3.2)) with  $\hat{Y}_S$  and  $Y_S$ ;

```

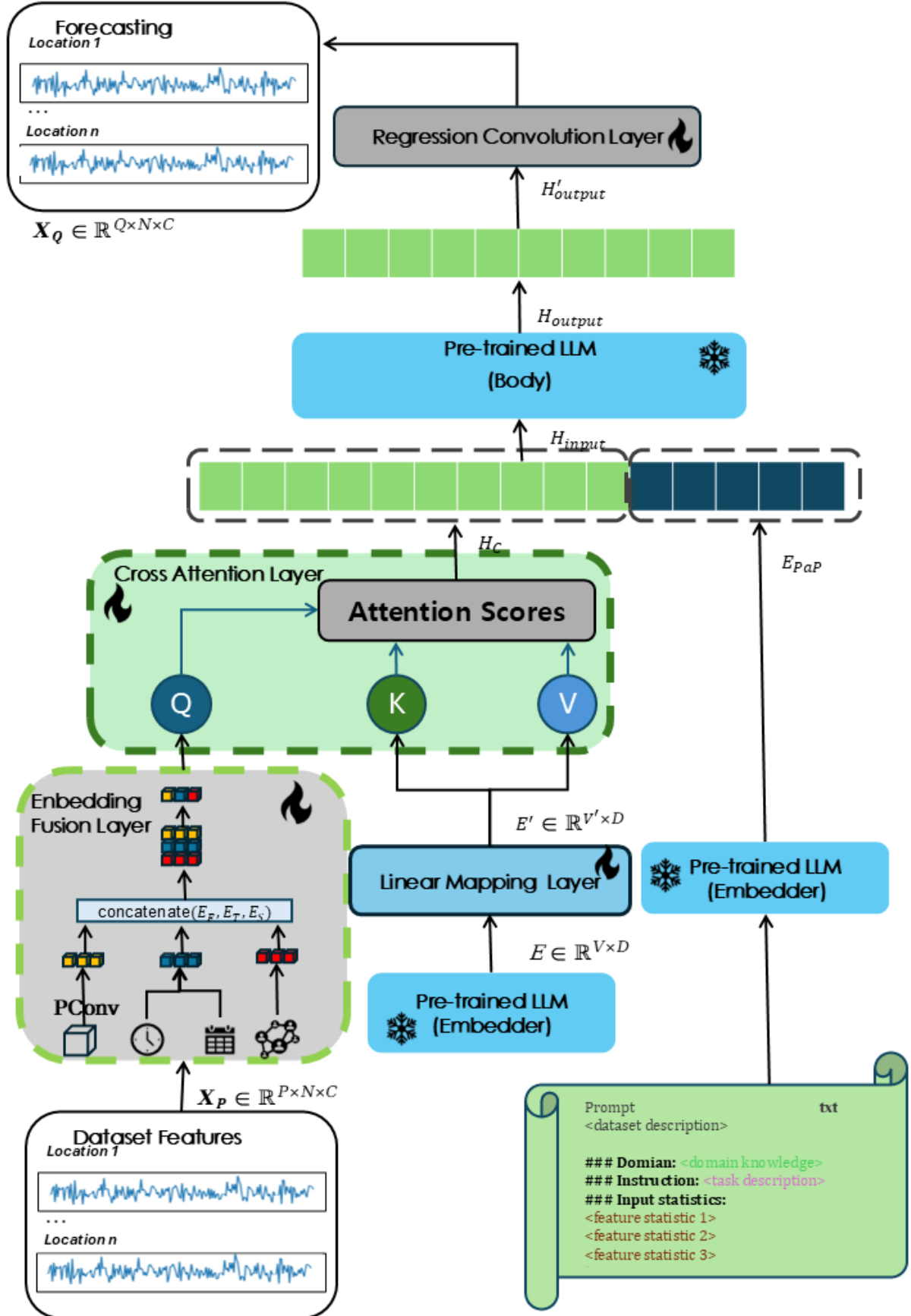


Figure 3.1: The STF-GPT2 framework, layers marked with a snowflake icon are frozen, meaning their parameters are excluded from gradient updates during training. Layers marked with a flame icon are trainable, and their parameters are updated via backpropagation.

4 Experiment

In this chapter, we conduct comprehensive experiments on STF-GPT2 to verify the superiority and feasibility of the model’s performance.

4.1 Dataset

As shown in Table 4.1, the **CitiBike** dataset consists of 2.6 million sharing bike orders in New York City from April 1 to June 30, 2016. Reflects the spatial and temporal usage patterns of shared bicycles. The dataset has been filtered to remove stations with few orders, and trips that started within the time span but ended outside it have also been excluded. The final dataset contains information from 250 stations with a total of 4,368 time steps, each representing a 30-minute interval.

The **NYCTaxi** dataset comprises 35 million taxi orders in New York City during the same period from April 1 to June 30, 2016, reflecting the flow of taxi traffic in both time and space. The data preprocessing procedure is similar to that of the CitiBike dataset, which includes 266 stations and 4 368 time steps, each of which also represents a 30-minute interval. The temporal alignment between the two datasets facilitates subsequent analyses.

Table 4.1: CitiBike and NYCTaxi Dataset Description

Dataset	CitiBike	NYCTaxi
Total	2.6 million	35 million
Number of Nodes	250	266
Time Span	01/04/2016 - 30/06/2016	01/04/2016 - 30/06/2016
Timestep	30 minutes	30 minutes
Number of Timesteps	4368	4368

4.2 Baseline

As baselines for comparative experiments with STF-GPT2, we select four Spatio-Temporal series prediction models based on large language models: OFA, GAT-GPT, ST-LLM, and GCN-GPT. Previous comparative studies have demonstrated that LLM-based models generally outperform traditional deep learning approaches in long-range Spatio-Temporal forecasting, few-shot, and zero-shot settings [9][30][17][16]. Therefore, in this study, we do not reproduce baseline results from conventional deep learning models.

Furthermore, recently proposed high-performing models, such as GPT4TFP, are not included due to the unavailability of open-source code, challenges in reproducing the methods, or limitations in computational resources.

- **OFA** [35]: Freezes the self-attention and feedforward network components within the transformer blocks of GPT-2 during training.
- **GAT-GPT** [2]: Combines Graph Attention Layers (GAT) with a frozen pre-trained GPT-2 Transformer to model spatial dependencies in the data.
- **ST-LLM** [16]: Integrates a pointwise convolution-based spatio-temporal fusion strategy with a frozen pre-trained GPT-2 Transformer to jointly capture spatial and temporal features.
- **ST-LLAMA2**: A variant of ST-LLM in which the GPT-2 backbone is replaced with the more advanced LLaMA2 model.
- **GCN-GPT** [16]: Combines Graph Convolutional Layers (GCN) with a frozen pre-trained GPT-2 Transformer to encode spatial correlations.

4.3 Evaluation Metrics

Due to the strong Spatio-Temporal characteristics, nonlinear fluctuations, and multiscale differences in traffic flow data, multiple dimensions are required to assess model performance. In addition to Mean Absolute Error (MAE), we also select Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and Weighted Absolute Percentage Error (WAPE). MAE and RMSE quantify absolute errors, while MAPE and

WAPE evaluate relative errors. In addition, all evaluation metrics in the experiments are averaged over all predicted time steps.

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{Y}_i - Y_i| \quad (4.1)$$

$$\text{MAPE} = \frac{100\%}{m} \sum_{i=1}^m \left| \frac{\hat{Y}_i - Y_i}{Y_i} \right| \quad (4.2)$$

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (\hat{Y}_i - Y_i)^2} \quad (4.3)$$

$$\text{WAPE} = \frac{\sum_{i=1}^m |\hat{Y}_i - Y_i|}{\sum_{i=1}^m |Y_i|} \times 100\% \quad (4.4)$$

where m is the number of all predicted values.

4.4 Implementations

Due to limited local computational resources and budget constraints, we selected the [Google Colab](#) cloud computing platform, which provides a single-core NVIDIA A100 GPU with 40GB of memory. As a Jupyter Notebook cloud platform, it lacks advanced IDE features and does not support multi-file project structures; therefore, all code was written and presented as scripts.

The CitiBike and NYCTaxi datasets were divided into training, validation, and test sets using the classic 6: 2: 2 ratio. Both the historical time steps length P and the future time step length S were set to 12, corresponding to 6 hours, to enable long-range Spatio-Temporal series forecasting.

All models were trained with the same default learning rate, early stopping tolerance, and number of epochs to ensure a fair comparison. For consistency in LLM configurations, OFA, GAT-GPT, and GCN-GPT were set with a 6-layer GPT2 structure [30]; ST-LLAMA2 was configured with an 8-layer LLAMA2 7B structure [16]; and STF-GPT2 employed a 2-layer GPT2 structure. The rationale behind the choice of layer numbers for STF-GPT2 is explained in the following section.

4.5 Model Optimization

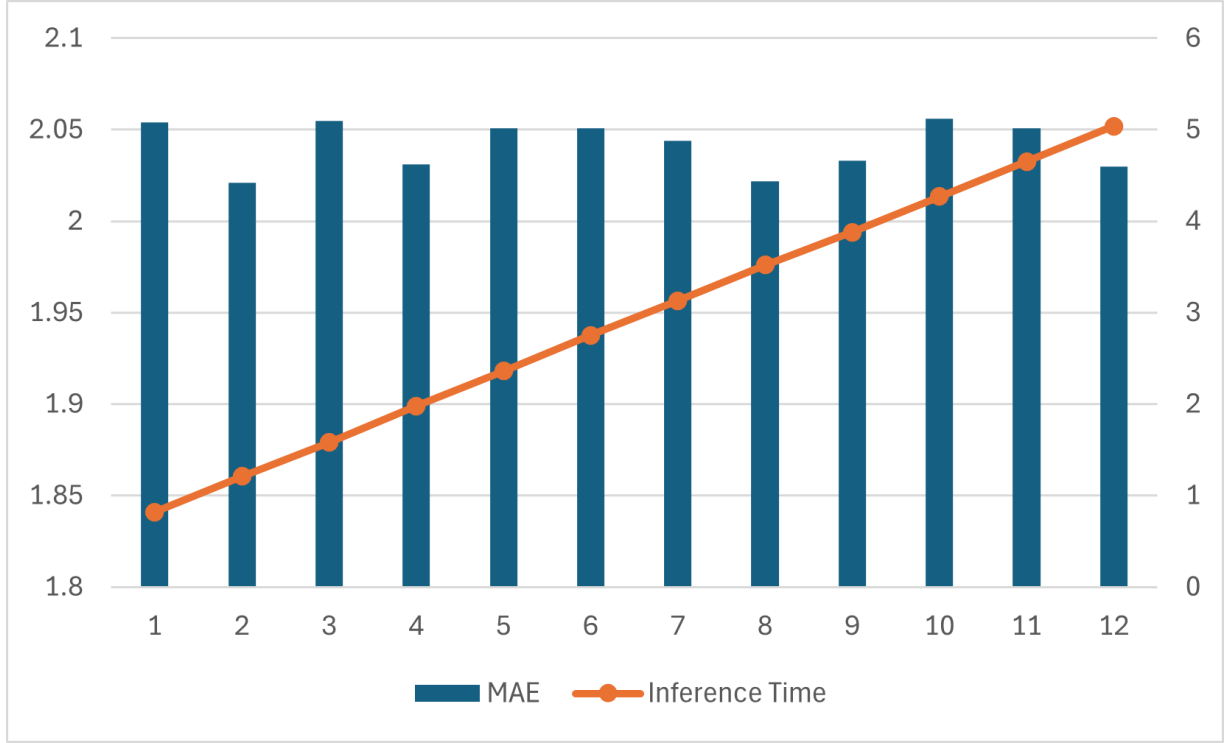


Figure 4.1: Performance evaluation of STF-GPT2 model with different Transformer layers in Citibike dataset. The left y-axis represents MAE; the right y-axis represents Inference Time (sec). The experimental data source is provided in Appendix A.

Before conducting a series of experiments, we systematically evaluate the impact of the number of transformer layers on the prediction performance of STF-GPT2. As one of the most critical hyperparameters in the model architecture, increasing the number of transformer layers theoretically allows the model to capture more complex spatial-temporal patterns. However, it also increases the number of parameters, leading to increased memory usage and higher training and inference costs. Moreover, for traffic flow data, which typically exhibits strong regularities—and downstream tasks with limited data, an excessive number of transformer layers may introduce a risk of overfitting. Deeper attention layers may focus on irrelevant information, and the training process becomes less stable. Therefore, we evaluate the performance (in terms of MAE) and resource consumption (inference time) of STF-GPT2 under different Transformer layer settings in both the NYC-Taxi and CitiBike datasets.

As shown in Fig. 4.1 and Fig. 4.2, we observe a linear increase in inference time as the number of Transformer layers increases, indicating an approximately linear relationship

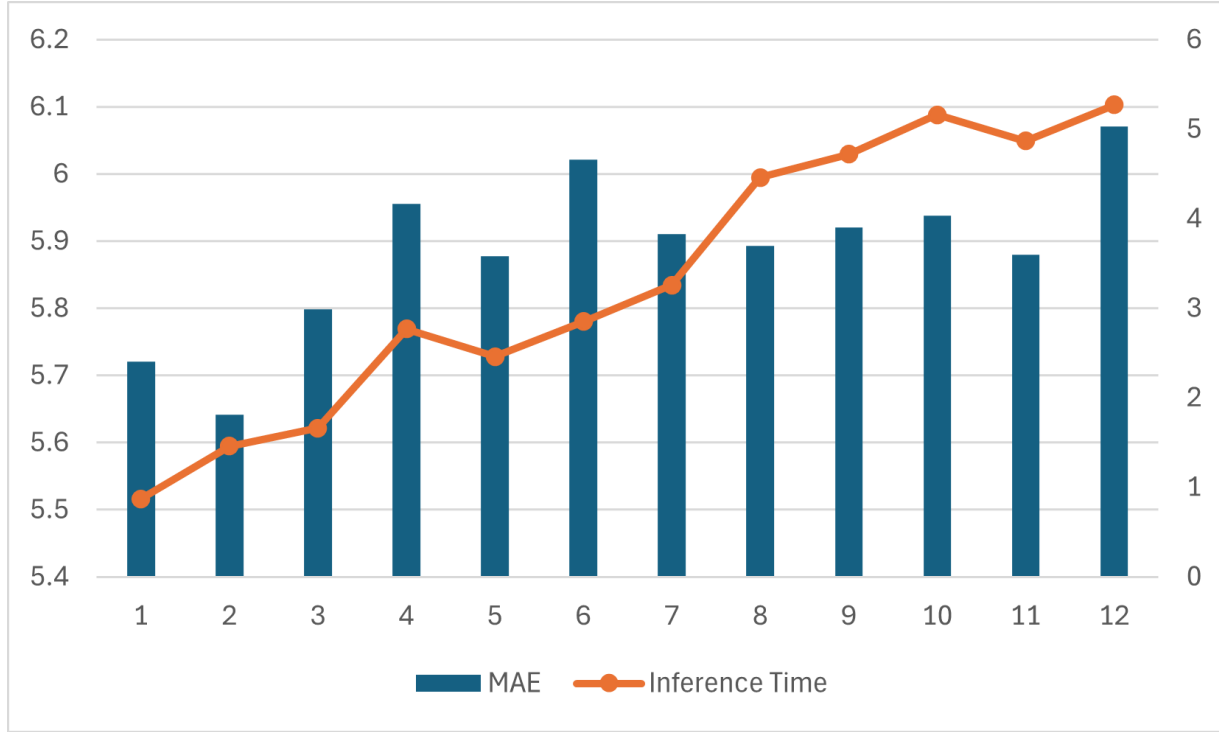


Figure 4.2: Performance evaluation of STF-GPT2 model with different Transformer layers in NYCTaxi dataset. The left y-axis represents MAE; the right y-axis represents Inference Time (sec). The experimental data source is provided in Appendix A.

between model computational overhead and layer depth. On the CitiBike dataset, the MAE shows only minor fluctuations across different layer counts. Although the lowest MAE is achieved with two layers, the performance differences across settings are relatively small. In contrast, the NYCTaxi dataset exhibits a more complex trend in MAE as the layer count changes: the error initially decreases and then increases, reaching its lowest value at two layers. However, the trend is not strictly monotonic and shows certain fluctuations.

These observations indicate that the effect of Transformer layer depth on model prediction performance is not linear, and architectural optimization requires careful balancing. Simply increasing the number of layers does not guarantee better performance, as one must weigh prediction accuracy against computational efficiency.

In conclusion, based on systematic evaluation, setting the number of transformer layers in STF-GPT2 to two achieves a favorable trade-off between prediction accuracy and inference efficiency. This setting results in both lower MAE and shorter inference time on the NYCTaxi and CitiBike datasets.

4.6 Ablation Study of Different Components

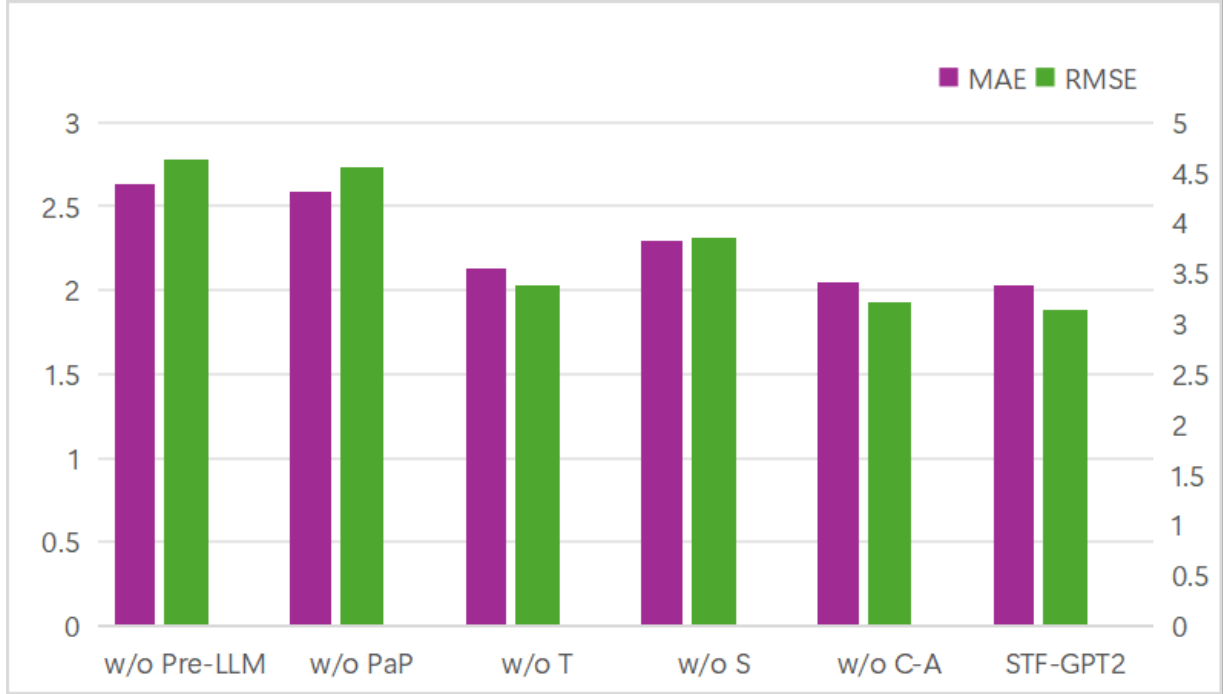


Figure 4.3: Performance ablation comparison of different variants of STF-GPT2 in CitiBike dataset. The left y-axis represents MAE; the right y-axis represents MAPE. The experimental data source is provided in Appendix A.

The STF-GPT2 model consists of multiple components, and ablation experiments are necessary to demonstrate the significant contributions of each component to the overall traffic prediction performance. In this section, we conduct an ablation study to evaluate the contribution of each major component in the proposed STF-GPT2 framework. Specifically, we remove the following modules one by one and compare the resulting performance against the full model baseline:

- **w/o Pre-LLM:** This variant removes the pre-trained large language model (LLM) layers entirely, allowing us to evaluate the importance of high-capacity language modeling for spatio-temporal forecasting. The model directly predicts outputs from the fused embeddings without Transformer-based contextualization.
- **w/o PaP:** The Prompt-as-Prefix strategy is removed in this variant. This setting examines the role of natural language instruction in guiding the model’s understanding and reasoning.

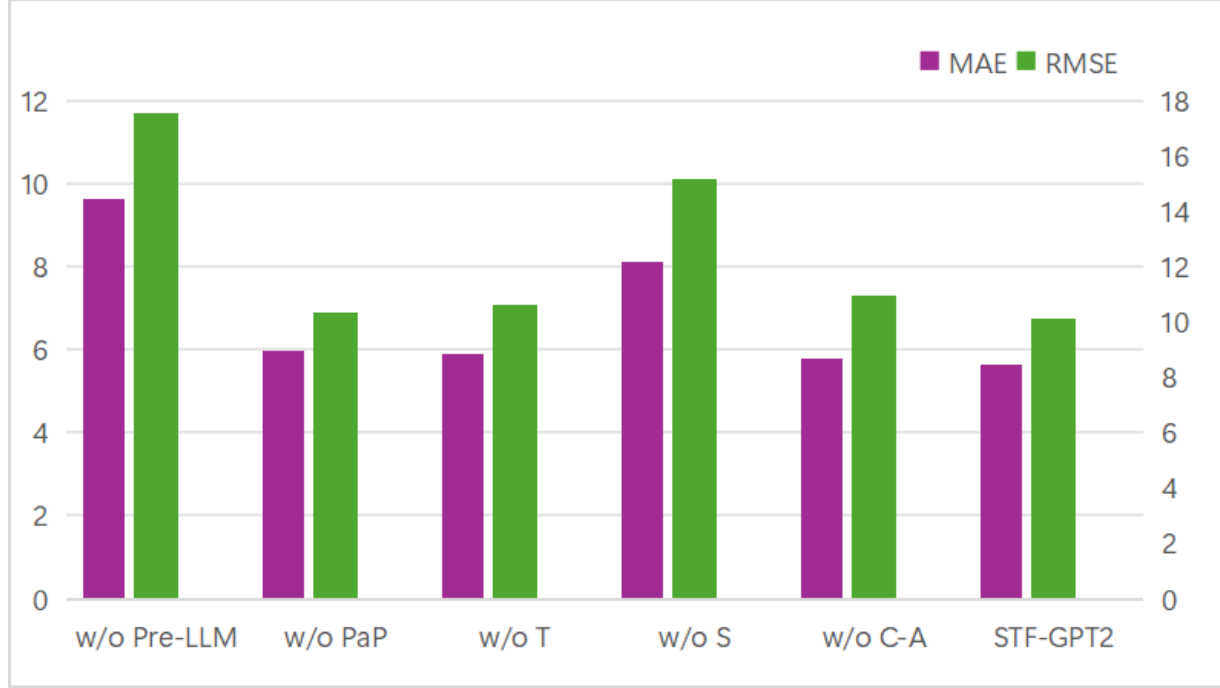


Figure 4.4: Performance ablation comparison of different variants of STF-GPT2 in NYCTaxi dataset. The left y-axis represents MAE; the right y-axis represents RMSE. The experimental data source is provided in Appendix A.

- **w/o Temp:** The temporal embedding module is excluded, and the model is trained without explicit temporal encoding, to investigate how much temporal context contributes to the overall performance.
- **w/o Spat:** This variant removes the spatial embedding component. It is used to assess the importance of weak spatial positional encoding in capturing spatial dependencies.
- **w/o C-A:** The cross-attention module used for modality alignment between spatial-temporal embeddings and the LLM input space is ablated in this variant, helping us evaluate the effectiveness of learned inter-modality interactions.

The completed STF-GPT2 model serves as the baseline for these ablation experiments. By comparing these variants, we aim to quantify the individual contribution of each architectural module to the model’s performance.

Fig. 4.3 and Fig. 4.4 presents the ablation results of the STF-GPT2 model in the CitiBike and NYCTaxi datasets, examining the impact of five key components on the overall model performance. The baseline corresponds to the complete STF-GPT2 model, while the five

variants, respectively, remove the pretrained large language model layer (w/o Pre-LLM), the Prompt-as-Prefix layer (w/o PaP), the temporal embedding layer (w/o T), the spatial embedding layer (w/o S), and the cross-attention modality alignment layer (w/o C-A).

Among them, the w/o Pre-LLM variant causes the most significant performance degradation on both datasets, reflected by substantial increases in MAE and RMSE metrics, indicating the strong dependence of STF-GPT2 on the pre-trained large language model. This component effectively captures complex Spatio-Temporal dependency patterns in traffic flow, and its removal notably weakens the model’s representational capability. Removing the Prompt-as-Prefix layer (w/o PaP) also leads to performance drops, especially more pronounced on the CitiBike dataset, suggesting that traffic flow patterns in the CitiBike scenario rely more heavily on semantic enhancement modeling guided by prompts in the large model. The removal of temporal embedding (w/o T) and spatial embedding (w/o S) layers both degrade performance, confirming the effectiveness of Spatio-Temporal embeddings in traffic forecasting tasks. Notably, on the NYCTaxi dataset, the absence of spatial embedding has a more pronounced negative impact, highlighting the dominant role of spatial structure in this scenario. The removal of the cross-attention modality alignment layer (w/o C-A) also leads to performance degradation, indicating that the cross-attention mechanism plays a crucial bridging role in aligning perceptual features with LLM embedding representations. Additionally, we observe that the error increase caused by removing the spatial embedding layer is greater than that caused by removing the temporal embedding layer, underscoring the importance of incorporating spatial structural features in Spatio-Temporal traffic flow prediction.

In summary, all components positively contribute to the model’s performance, especially the collaborative modeling capability between spatial structure and the pre-trained language model. On both datasets, the full STF-GPT2 model achieves the best prediction results, demonstrating that its design integrating spatial and temporal embeddings, pretrained language models, the Prompt-as-Prefix strategy, and the modality alignment mechanism significantly enhances Spatio-Temporal series modeling ability and exhibits superior generalization performance in complex traffic scenarios.

5 Results

5.1 Comparative Experiments

Table 5.1: Comparison of experimental performance evaluation results. Bolding indicates that the metric is optimal. The experimental data source is provided in Appendix A.

	OFA	GAT-GPT	GCN-GPT	ST- LLAMA2	ST-GPT2
Metric	CitiBike Pick-up				
MAE	2.06	2.07	2.37	2.10	2.02
RMSE	3.21	3.23	3.80	3.37	3.14
MAPE	53.55%	52.54%	56.24%	56.63%	55.27%
WAPE	41.70%	41.70%	47.66%	42.49%	40.88%
Metric	CitiBike Drop-off				
MAE	1.96	1.95	2.24	1.99	1.92
RMSE	2.97	2.94	3.48	3.03	2.87
MAPE	49.64%	49.26%	51.05%	55.23%	50.14%
WAPE	39.68%	39.43%	45.37%	40.28%	38.91%
Metric	NYCTaxi Pick-up				
MAE	5.82	5.92	6.58	5.35	5.64
RMSE	10.42	10.55	12.23	9.48	10.14
MAPE	36.67%	37.83%	40.19%	41.32%	36.24%
WAPE	22.00%	22.39%	24.88%	20.27%	21.35%
Metric	NYCTaxi Drop-off				
MAE	5.60	5.66	6.64	5.66	5.48
RMSE	10.14	10.39	12.24	10.74	10.14
MAPE	37.39%	37.36%	42.46%	47.47%	36.75%
WAPE	21.36%	21.60%	25.32%	21.63%	20.94%
1st Count	1	2	0	3	11

We conduct comparative experiments on the complete NYCTaxi and CitiBike datasets to comprehensively evaluate the performance of our STF-GPT2 model against four baseline models: OFA, GAT-GPT, ST-LLM, and GCN-GPT. The evaluation metrics include MAE,

RMSE, MAPE, and WAPE, where lower values indicate better performance. In the table, the best performance results are highlighted in bold font.

From Table. 5.1, we observe that the STF-GPT2 model demonstrates significant advantages across all tasks and metrics, achieving first place in 11 out of 16 indicators. It shows strong predictive performance and generalization ability across various traffic scenarios. Compared to OFA and ST-LLAMA2, STF-GPT2 consistently outperforms them on all evaluation metrics, especially in the NYCTaxi Drop-off and CitiBike-related tasks, where STF-GPT2 attains the lowest MAE and RMSE. This indicates its superior capability to capture microscopic fluctuations and Spatio-Temporal structures in different traffic environments. This advantage likely stems from the Spatio-Temporal embedding mechanism and the Prompt-as-Prefix fusion strategy introduced in STF-GPT2, which enhance the model’s understanding of both local and global dynamic patterns.

Although GAT-GPT and GCN-GPT incorporate graph structural information and LLM encoding capabilities in their design, their overall accuracy still lags behind STF-GPT2 despite performing reasonably well on some tasks. This suggests that simply combining graph neural mechanisms with language models is insufficient to fully unleash the potential of LLMs in modeling traffic Spatio-Temporal dependencies. The lack of an efficient modality collaboration mechanism may be their main limitation.

Notably, ST-LLAMA2 achieves prediction performance close to STF-GPT2 on multiple tasks but overall remains slightly inferior, particularly showing higher errors on the CitiBike dataset. We speculate that ST-LLAMA2 introduces deeper Transformer encoding modules, which, while expressive, tend to overfit local features when dealing with datasets like CitiBike that have relatively regular Spatio-Temporal structures and more distinct patterns.

In summary, compared with existing LLM-based traffic prediction models, STF-GPT2 leads in both prediction accuracy and stability. Its multi-component collaborative design demonstrates stronger generalization and robustness in handling complex Spatio-Temporal traffic series problems.

5.2 Few-Shot Forecasting

We cropped the original training set to retain only 10% of the training data , to simulate the Few-shot scenario where the data source is scarce. Then we our STF-GPT2 model

Table 5.2: Comparison of experimental performance evaluation results in Few-Shot forecasting. Bolding indicates that the metric is optimal. The experimental data source is provided in Appendix A.

	OFA	GAT-GPT	GCN-GPT	ST-LLAMA2	ST-GPT2
Metric	CitiBike Pick-up				
MAE	2.20	2.59	2.73	2.24	2.16
RMSE	3.59	4.41	4.44	3.58	3.44
MAPE	57.52%	56.23%	56.93%	59.47%	58.31%
WAPE	44.40%	52.20%	55.20%	45.20%	43.58%
Metric	CitiBike Drop-off				
MAE	2.06	2.50	2.79	2.11	2.04
RMSE	3.17	4.07	4.65	3.23	3.12
MAPE	55.96%	56.36%	61.85%	54.44%	54.84%
WAPE	41.63%	50.64%	56.28%	42.75%	41.38%
Metric	NYCTaxi Pick-up				
MAE	6.49	7.02	10.31	5.81	5.88
RMSE	12.12	13.09	18.82	10.16	10.70
MAPE	46.74%	50.19%	59.41%	41.82%	38.06%
WAPE	24.54%	26.54%	39.02%	21.99%	22.27%
Metric	NYCTaxi Drop-off				
MAE	6.27	6.84	9.25	5.59	5.68
RMSE	12.10	13.27	19.50	9.90	10.95
MAPE	45.23%	56.15%	56.77%	40.58%	37.42%
WAPE	23.92%	26.09%	35.28%	21.35%	21.68%
1st Count	0	1	0	7	8

with four baseline models for a comprehensive performance evaluation.

From Table 5.2, we observe that the ST-GPT2 model demonstrates excellent performance and stable robustness across multiple datasets, particularly under few-shot forecasting scenarios. On the CitiBike pick-up and drop-off datasets, ST-GPT2 achieves the lowest MAE values of 2.16 and 2.04, and RMSE values of 3.44 and 3.12, respectively. These results indicate that ST-GPT2 not only outperforms OFA, GAT-GPT, and GCN-GPT, but also slightly surpasses the state-of-the-art ST-LLAMA2 in terms of absolute and squared error. Although ST-LLAMA2 exhibits a marginally lower MAPE on the drop-off dataset (54.44% vs. 54.84%), ST-GPT2 consistently achieves the lowest WAPE scores (43.58% and 41.38%), highlighting its superior ability to control overall prediction error. On the NYCTaxi pick-up and drop-off datasets, ST-GPT2 maintains competitive performance. It obtains MAE values of 5.88 and 5.68, and RMSE values of 10.70 and 10.95, respectively very close to ST-LLAMA2 (5.81 and 5.59 for MAE, 10.16 and 9.90 for RMSE). Notably, ST-GPT2 surpasses all baselines in terms of relative error, achieving the lowest MAPE (38.06% and 37.42%) and maintaining WAPE values (22.27% and 21.68%) on par with or better than ST-LLAMA2. These results confirm the model’s capacity to generalize effectively even on the different traffic flow datasets. Overall, ST-GPT2 attains the best metric in 8 out of 16 evaluation metrics, the highest among all compared models. It consistently delivers low absolute and relative errors across all datasets, demonstrating not only strong predictive accuracy but also robust generalization in few-shot and limited-data scenario.

5.3 Zero-Shot Forecasting

We design a series of zero-shot prediction experiments to evaluate the model’s ability to generalize across different tasks within the same city, without any fine-tuning on the target task. Specifically, we focus on intra-domain transfer scenarios where models trained on one traffic prediction task are directly evaluated on another task from the same domain. The evaluated tasks are:

- **Taxi Pick-up $\rightarrow$ Taxi Drop-off:** The model is trained on the NYCTaxi Pick-up dataset and evaluated on the NYCTaxi Drop-off dataset for traffic flow forecasting.
- **Taxi Drop-off $\rightarrow$ Taxi Pick-up:** The model is trained on the NYCTaxi Drop-off dataset and evaluated on the NYCTaxi Pick-up dataset.

Table 5.3: Comparison of experimental performance evaluation results in Zero-Shot forecasting. Bolding indicates that the metric is optimal. The experimental data source is provided in Appendix A.

	GAT-GPT	GCN-GPT	ST- LLM	ST-GPT2
Metric	Taxi Pick-up → Taxi Drop-off			
MAE	10.00	11.03	9.31	10.75
RMSE	21.16	21.86	18.68	20.88
MAPE	68.03%	70.32%	66.42%	79.49%
Metric	Taxi Drop-off → Taxi Pick-up			
MAE	9.67	8.09	8.02	10.37
RMSE	17.76	14.58	13.21	20.11
MAPE	73.46%	50.99%	46.16%	69.31%
Metric	Bike Drop-off → Bike Pick-up			
MAE	2.49	2.54	2.41	2.23
RMSE	4.24	4.29	4.16	4.08
MAPE	60.14%	58.29%	60.44%	58.06%
Metric	Bike Pick-up → Bike Drop-off			
MAE	2.4	2.24	2.41	2.03
RMSE	3.96	3.48	3.96	3.14
MAPE	61.97%	51.05%	62.57%	55.01%
1st Count	0	0	6	6

- **Bike Drop-off $\rightarrow$ Bike Pick-up:** The model is trained on the CitiBike Drop-off dataset and evaluated on the CitiBike Pick-up dataset.
- **Bike Pick-up $\rightarrow$ Bike Drop-off:** The model is trained on the CitiBike Pick-up dataset and evaluated on the CitiBike Drop-off dataset.

As shown in Table 5.3, both ST-LLM and our proposed ST-GPT2 demonstrate strong zero-shot transfer capabilities, achieving the best results in six metrics respectively across all four tasks. Particularly, in key error metrics such as MAE and RMSE, both models significantly outperform baseline methods like GAT-GPT and GCN-GPT, which incorporate graph structures but lack Spatio-Temporal fusion mechanisms.

More specifically, ST-LLM excels in the Taxi-related tasks, achieving the lowest errors in both the Pick-up $\rightarrow$ Drop-off and Drop-off $\rightarrow$ Pick-up directions. This suggests that ST-LLM is well-suited for modeling macro-level urban traffic patterns. On the other hand, ST-GPT2 shows outstanding performance in the Bike-related tasks, especially in the Drop-off $\rightarrow$ Pick-up and Pick-up $\rightarrow$ Drop-off scenarios, where it achieves the best MAE and RMSE. This indicates that ST-GPT2 is more effective in capturing short-term dynamics at the micro-level. Furthermore, GAT-GPT and GCN-GPT fail to achieve optimal results in any of the tasks, highlighting the limitations of relying solely on graph structures without effective spatio-temporal integration. In conclusion, both ST-LLM and our ST-GPT2 achieve strong zero-shot performance across multiple intra-domain transfer tasks, validating the high compatibility between our proposed Spatio-Temporal fusion strategy and pre-trained language models.

6 Conclusions

This study empirically demonstrates the feasibility of using STF-GPT2 for Spatio-Temporal series forecasting on traffic flow datasets. Under conditions of full data availability, few-shot, and zero-shot settings, STF-GPT2 consistently outperforms several mainstream LLM-based reprogramming models on the CitiBike and NYCTaxi datasets. These results highlight the superior generalization and knowledge transfer capabilities of STF-GPT2 compared to existing approaches.

Despite incorporating a relatively weak spatial dependency mechanism in its spatial embedding layer, STF-GPT2 still achieves better performance on CitiBike and NYCTaxi datasets with relatively simple spatial structures than models like GCN-GPT and GAT-GPT, which utilize more sophisticated spatial modules. However, this design choice may limit STF-GPT2 effectiveness on datasets with more complex spatial dependencies, where models like GCN-GPT and GAT-GPT could potentially outperform it.

Future work will involve evaluating STF-GPT2 on a broader range of Spatio-Temporal traffic datasets to validate this hypothesis. In addition, we plan to investigate the performance impact of replacing GPT-2 with other pretrained LLMs, such as LLaMA2 or BERT, to further explore and identify the most suitable foundation model for Spatio-Temporal forecasting tasks.

Bibliography

- [1] S. Y. Chang, H.-C. Wu, and Y.-C. Kao. “Tensor extended Kalman filter and its application to traffic prediction”. In: *IEEE Transactions on Intelligent Transportation Systems* 24.12 (2023), pp. 13813–13829.
- [2] Y. Chen, X. Wang, and G. Xu. *GATGPT: A Pre-trained Large Language Model with Graph Attention Network for Spatiotemporal Imputation*. 2023. arXiv: [2311.14332](https://arxiv.org/abs/2311.14332) [cs.LG]. URL: <https://arxiv.org/abs/2311.14332>.
- [3] X. Cui and H. Lv. “STEFT: Spatio-Temporal Embedding Fusion Transformer for Traffic Prediction”. In: *Electronics* 13.19 (2024), p. 3816.
- [4] S. Ganesan and D. Subramani. “Spatio-temporal predictive modeling framework for infectious disease spread”. In: *Scientific Reports* 11.1 (2021), p. 6741.
- [5] L. Huang, F. Mao, K. Zhang, and Z. Li. “Spatial-temporal convolutional transformer network for multivariate time series forecasting”. In: *Sensors* 22.3 (2022), p. 841.
- [6] X. Huang. *Enhancing Traffic Prediction with Textual Data Using Large Language Models*. 2024. arXiv: [2405.06719](https://arxiv.org/abs/2405.06719) [cs.CL]. URL: <https://arxiv.org/abs/2405.06719>.
- [7] A. Jaegle, S. Borgeaud, J.-B. Alayrac, C. Doersch, C. Ionescu, D. Ding, S. Koppula, D. Zoran, A. Brock, E. Shelhamer, O. Hénaff, M. M. Botvinick, A. Zisserman, O. Vinyals, and J. Carreira. *Perceiver IO: A General Architecture for Structured Inputs Outputs*. 2022. arXiv: [2107.14795](https://arxiv.org/abs/2107.14795) [cs.LG]. URL: <https://arxiv.org/abs/2107.14795>.
- [8] G. Jin, Y. Liang, Y. Fang, Z. Shao, J. Huang, J. Zhang, and Y. Zheng. “Spatio-temporal graph neural networks for predictive learning in urban computing: A survey”. In: *IEEE Transactions on Knowledge and Data Engineering* 36.10 (2023), pp. 5388–5408.
- [9] M. Jin, S. Wang, L. Ma, Z. Chu, J. Y. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-F. Li, S. Pan, and Q. Wen. *Time-LLM: Time Series Forecasting by Reprogramming Large Language Models*. 2024. arXiv: [2310.01728](https://arxiv.org/abs/2310.01728) [cs.LG]. URL: <https://arxiv.org/abs/2310.01728>.

- [10] D. Ko, J. Choi, H. K. Choi, K.-W. On, B. Roh, and H. J. Kim. “Meltr: Meta loss transformer for learning to fine-tune video foundation models”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 20105–20115.
- [11] S. V. Kumar and L. Vanajakshi. “Short-term traffic flow prediction using seasonal ARIMA model with limited input data”. In: *European Transport Research Review* 7 (2015), pp. 1–9.
- [12] H. Larochelle, D. Erhan, and Y. Bengio. “Zero-data learning of new tasks.” In: *AAAI*. Vol. 1. 2. 2008, p. 3.
- [13] H. Li, J. Liu, S. Han, J. Zhou, T. Zhang, and C. Philip Chen. “STFGCN: Spatial-temporal fusion graph convolutional network for traffic prediction”. In: *Expert Systems with Applications* 255 (2024), p. 124648. ISSN: 0957-4174. DOI: <https://doi.org/10.1016/j.eswa.2024.124648>. URL: <https://www.sciencedirect.com/science/article/pii/S095741742401515X>.
- [14] Y. Li, R. Yu, C. Shahabi, and Y. Liu. *Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting*. 2018. arXiv: [1707.01926](https://arxiv.org/abs/1707.01926) [cs.LG]. URL: <https://arxiv.org/abs/1707.01926>.
- [15] Y. Liang, G. Huang, and Z. Zhao. *Bike Sharing Demand Prediction based on Knowledge Sharing across Modes: A Graph-based Deep Learning Approach*. 2022. arXiv: [2203.10961](https://arxiv.org/abs/2203.10961) [cs.LG]. URL: <https://arxiv.org/abs/2203.10961>.
- [16] C. Liu, S. Yang, Q. Xu, Z. Li, C. Long, Z. Li, and R. Zhao. “Spatial-temporal large language model for traffic prediction”. In: *2024 25th IEEE International Conference on Mobile Data Management (MDM)*. IEEE. 2024, pp. 31–40.
- [17] L. Liu, S. Yu, R. Wang, Z. Ma, and Y. Shen. *How Can Large Language Models Understand Spatial-Temporal Data?* 2024. arXiv: [2401.14192](https://arxiv.org/abs/2401.14192) [cs.LG]. URL: <https://arxiv.org/abs/2401.14192>.
- [18] Y. Liu, G. Qin, X. Huang, J. Wang, and M. Long. *AutoTimes: Autoregressive Time Series Forecasters via Large Language Models*. 2024. arXiv: [2402.02370](https://arxiv.org/abs/2402.02370) [cs.LG]. URL: <https://arxiv.org/abs/2402.02370>.
- [19] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: [2103.00020](https://arxiv.org/abs/2103.00020) [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.

- [20] A. Ramezani and Y. Xu. “Knowledge of cultural moral norms in large language models”. In: *arXiv preprint arXiv:2306.01857* (2023).
- [21] L. She, C. Zhang, X. Man, and J. Shao. “LLMDiff: Diffusion Model Using Frozen LLM Transformers for Precipitation Nowcasting”. In: *Sensors* 24.18 (2024), p. 6049.
- [22] B. Shen, X. Liang, Y. Ouyang, M. Liu, W. Zheng, and K. M. Carley. “Stepdeep: A novel spatial-temporal mobility event prediction framework based on deep neural network”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2018, pp. 724–733.
- [23] S. S. Sofi and I. Oseledets. “A case study of spatiotemporal forecasting techniques for weather forecasting”. In: *GeoInformatica* 29.2 (Sept. 2024), pp. 275–298. ISSN: 1573-7624. DOI: [10.1007/s10707-024-00530-y](https://doi.org/10.1007/s10707-024-00530-y). URL: <http://dx.doi.org/10.1007/s10707-024-00530-y>.
- [24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. *Graph Attention Networks*. 2018. arXiv: [1710.10903](https://arxiv.org/abs/1710.10903) [stat.ML]. URL: <https://arxiv.org/abs/1710.10903>.
- [25] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra. *Matching Networks for One Shot Learning*. 2017. arXiv: [1606.04080](https://arxiv.org/abs/1606.04080) [cs.LG]. URL: <https://arxiv.org/abs/1606.04080>.
- [26] S. Wang, H. Miao, H. Chen, and Z. Huang. “Multi-task adversarial spatial-temporal networks for crowd flow prediction”. In: *Proceedings of the 29th ACM international conference on information & knowledge management*. 2020, pp. 1555–1564.
- [27] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun. “Transformers in time series: A survey”. In: *arXiv preprint arXiv:2202.07125* (2022).
- [28] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang. *Graph WaveNet for Deep Spatial-Temporal Graph Modeling*. 2019. arXiv: [1906.00121](https://arxiv.org/abs/1906.00121) [cs.LG]. URL: <https://arxiv.org/abs/1906.00121>.
- [29] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang. “Graph wavenet for deep spatial-temporal graph modeling”. In: *arXiv preprint arXiv:1906.00121* (2019).
- [30] Y. Xu and M. Liu. “GPT4TFP: Spatio-temporal fusion large language model for traffic flow prediction”. In: *Neurocomputing* 625 (2025), p. 129562. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2025.129562>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231225002346>.

- [31] C.-H. H. Yang, Y.-Y. Tsai, and P.-Y. Chen. *Voice2Series: Reprogramming Acoustic Models for Time Series Classification*. 2022. arXiv: [2106.09296](https://arxiv.org/abs/2106.09296) [cs.LG]. URL: <https://arxiv.org/abs/2106.09296>.
- [32] Z. Yuan, X. Zhou, and T. Yang. “Hetero-convlstm: A deep learning approach to traffic accident prediction on heterogeneous spatio-temporal data”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2018, pp. 984–992.
- [33] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff. *A Transformer-based Framework for Multivariate Time Series Representation Learning*. 2020. arXiv: [2010.02803](https://arxiv.org/abs/2010.02803) [cs.LG]. URL: <https://arxiv.org/abs/2010.02803>.
- [34] P. Zhao and H. Hu. “Geographical patterns of traffic congestion in growing megacities: Big data analytics from Beijing”. In: *Cities* 92 (2019), pp. 164–174.
- [35] T. Zhou, P. Niu, X. Wang, L. Sun, and R. Jin. “One fits all: power general time series analysis by pretrained LM”. In: *Proceedings of the 37th International Conference on Neural Information Processing Systems*. NIPS ’23. New Orleans, LA, USA: Curran Associates Inc., 2023.

Appendix A Experiment Result Data

All experimental results reported in this paper are based on publicly accessible datasets and our custom experimental CSV files. The full version of the processed data used in our experiments can be found at the following GitHub repository:

- **GitHub Repository:** <https://github.com/Sirius-CJackie/SFT-GPT2>

The directory structure of the `Experiment_result` folder:

```
| - Ablation_experiment/  
|   | - Ablation_experiment.xlsx  
| - Layers_performance/  
|   | - Number of model layers and performance.xlsx  
| - STF-GPT2/  
|   | - bike_drop/  
|     | - test.csv  
|   | - bike_pick/  
|     | - test.csv  
|   | - few-shot_bike_drop/  
|     | - test.csv  
|   | - few-shot_bike_pick/  
|     | - test.csv  
|   | - few-shot_taxi_drop/  
|     | - test.csv  
|   | - few-shot_taxi_pick/  
|     | - test.csv  
|   | - taxi_drop/  
|     | - test.csv  
|   | - taxi_pick/  
|     | - test.csv  
|   | - zero-shot_taxi_drop_pick/  
|     | - test.csv  
|   | - zero-shot_taxi_pick_drop/  
|     | - test.csv
```

```
| - Zero-Shot_Bike_dataset/
|   | - zero-shot_citibike.xls
```

The experimental results of STF-GPT2 reported in Table 5.1 are based on the following datasets:

- STF-GPT2 on the Citibike pick-up dataset: `STF-GPT2/bike_pick/test.csv`
- STF-GPT2 on the Citibike drop-off dataset: `STF-GPT2/bike_drop/test.csv`
- STF-GPT2 on the Taxi pick-up dataset: `STF-GPT2/taxi_pick/test.csv`
- STF-GPT2 on the Taxi drop-off dataset: `STF-GPT2/taxi_drop/test.csv`

For all other models on the Citibike and NYCTaxi datasets, the experimental results are directly cited from the ST-LLM paper [16], as their experimental settings are entirely consistent with those used in this work.

The experimental results of STF-GPT2 reported in Table 5.2 in the few-shot setting are based on:

- STF-GPT2 on the Citibike pick-up dataset: `STF-GPT2/few-shot_bike_pick/test.csv`
- STF-GPT2 on the Citibike drop-off dataset: `STF-GPT2/few-shot_bike_drop/test.csv`
- STF-GPT2 on the Taxi pick-up dataset: `STF-GPT2/few-shot_taxi_pick/test.csv`
- STF-GPT2 on the Taxi drop-off dataset: `STF-GPT2/few-shot_taxi_drop/test.csv`

The results of other models under the few-shot setting are also referenced from the ST-LLM paper [16], due to identical experimental configurations.

In Table 5.3, the experimental data of STF-GPT2 for Taxi Pick-up $\rightarrow$ Taxi Drop-off task is sourced from `STF-GPT2/zero-shot_taxi_pick_drop/test.csv`; the data for Taxi Drop-off $\rightarrow$ Taxi Pick-up task is sourced from `STF-GPT2/zero-shot_taxi_drop_pick/test.csv`. The experimental data for GAT-GPT, GCN-GPT, ST-LLM, and STF-GPT2 on Bike Drop-off $\rightarrow$ Bike Pick-up and Bike Pick-up $\rightarrow$ Bike Drop-off tasks are all from `Zero-Shot_Bike_dataset/zero-shot_citibike.xls`. All other experimental results in

this table are directly cited from the ST-LLM paper [16], as their experimental settings are completely consistent with those in this work.

The experimental data for Fig. 4.1 and Fig. 4.2 are sourced from `Layers_performance/Number of model layers and performance.xlsx`; the data for Fig. 4.3 and Fig. 4.4 are sourced from `Ablation_experiment/Ablation_experiment.xlsx`.

Statement: We acknowledge that a significant portion of the experimental results cited in this paper are taken directly from the ST-LLM paper [16], and we did not re-implement these experiments ourselves. The primary reason is the high computational cost associated with reproducing large language model experiments — each training run on a single dataset typically requires approximately two hours on an A100 GPU. As mentioned previously, we relied on self-funded cloud computing resources, which are prohibitively expensive given our limited budget. Furthermore, since the experimental settings in our work are identical to those in the ST-LLM paper, we verified that the partially reproduced results closely match those reported by ST-LLM. Therefore, to conserve resources, we chose to directly cite their published results. We sincerely apologize for any inconvenience this may cause.

Appendix B Reproduce Project

Our project has been uploaded to [GitHub](#). Since it was developed using Jupyter Notebook, the deployment process is straightforward. We recommend executing the code directly on Google Colab for ease of use.

Dependencies:

- Python 3.8.19
- PyTorch 2.4.1
- CUDA 11.7

Datasets: The datasets used in our experiments are included in the project directory under the folders `CitiBike_dataset/` and `NYCTaxi_dataset/`.

Running Instructions: Before training, please configure the following training dictionary:

```
config = {  
    "file_dir": "CitiBike_dataset/bike_pick/",  
    "epochs": 300,  
    "num_nodes": 250,  
    "lr": 1e-3,  
    "batch_size": 64,  
    "input_dim": 3,  
    "clip": 5.0,  
    "es_patience": 100,  
    "input_len": 12,  
    "output_len": 12,  
    "wdecay": 0.0001,  
    "save": "./",  
}
```


- `file_dir`: set the path to the dataset folder (either `CitiBike_dataset` or `NYCTaxi_dataset`).
- `epochs`: number of training epochs.
- `num_nodes`: number of spatial IDs in the dataset (250 for CitiBike, 266 for NYC-Taxi).
- `lr`: learning rate.
- `batch_size`: batch size, default is 64.
- `input_dim`: number of input features, default is 3.
- `clip`: gradient clipping norm, default is 5.0.
- `es_patience`: early stopping patience, training stops if no improvement is observed for 100 epochs.
- `input_len`: input series length.
- `output_len`: prediction series length.
- `wdecay`: weight decay coefficient for regularization.
- `save`: directory to save output files.

Appendix C Use of Language and Writing Assistance Tools

In the preparation of this thesis, GPT-based tools were utilized to improve the clarity, coherence, and grammar of the manuscript. Specifically, GPT was employed for semantic and syntactic refinement of the text, correction of paragraph structure, and adjustments to LaTeX formatting. Additionally, GPT was consulted for conceptual clarification during the research process. All content and interpretations remain the sole responsibility of the author.